

ALBERI AVL

PIETRO DI LENA

DIPARTIMENTO DI INFORMATICA – SCIENZA E INGEGNERIA
UNIVERSITÀ DI BOLOGNA

ALGORITMI E STRUTTURE DI DATI
ANNO ACCADEMICO 2024/2025



INTRODUZIONE

- Abbiamo visto che, data una chiave k , con un BST è possibile inserire, rimuovere e cercare nodi in tempo $O(h)$, dove h è l'altezza dell'albero
 - Un albero binario con n nodi ha altezza $h = \Omega(\log n)$ e $h = O(n)$
 - Problema: scrivere un algoritmo che prese in input n chiavi costruisca un BST con altezza $h = \Theta(n)$
 - Problema: scrivere un algoritmo che prese in input n chiavi costruisca un BST con altezza $h = \Theta(\log n)$
- Inserimenti e rimozioni possono **sbilanciare** l'albero
 - Un albero sbilanciato ha al peggio altezza lineare sul numero di nodi
- Obiettivo: **mantenere bilanciato un BST**, anche a seguito di inserimenti e rimozioni di nodi, in modo da avere operazioni di inserimento, rimozione e ricerca con **costo pessimo logaritmico**

ALBERI AVL (ADELSON-VELSKY E LANDIS, 1962)

- Un **albero AVL** è un albero binario (quasi) perfettamente bilanciato
 - Il nome deriva dagli autori **Adelson-Velsky** e **Landis**
- Un albero AVL con n nodi supporta le operazioni di base SEARCH, INSERT, DELETE con costo $O(\log n)$
- In un albero AVL le operazioni di inserimento e rimozione sono implementate in modo da essere **auto-bilancianti**
 - L'albero viene automaticamente ribilanciato in seguito ad inserimenti e rimozioni che causino sbilanciamento

Fattore di bilanciamento

Il **fattore di bilanciamento** $\beta(v)$ di un nodo v è dato dalla differenza tra l'altezza del sottoalbero sinistro e destro di v :

$$\beta(v) = \text{altezza}(v.\text{left}) - \text{altezza}(v.\text{right})$$

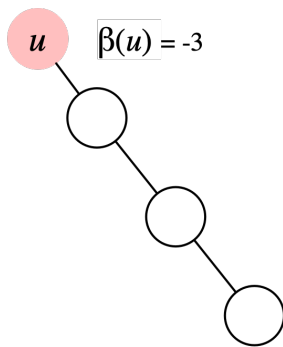
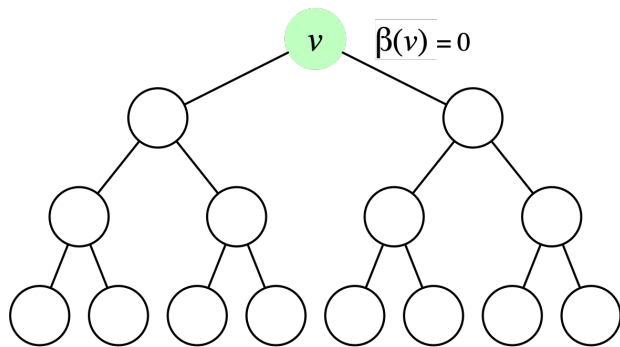
Bilanciamento in altezza

Un albero si dice **bilanciato in altezza** se per ogni nodo v le altezze dei suoi sottoalberi sinistro ($v.\text{left}$) e destro ($v.\text{right}$) differiscono al più di uno

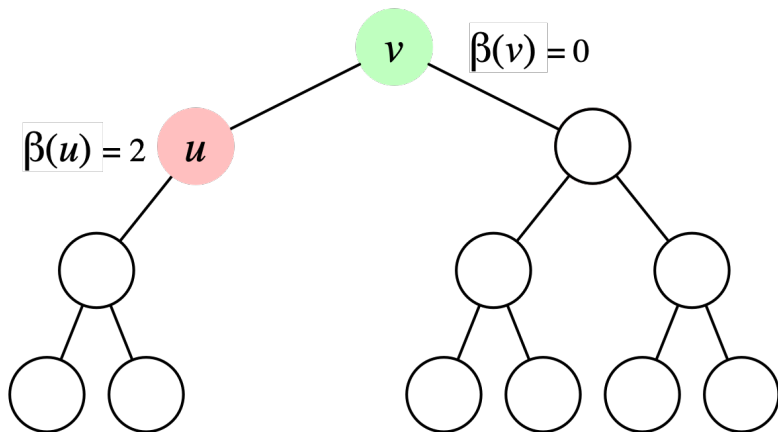
$$|\beta(v)| \leq 1$$

- Nota. Per definizione, l'altezza di un albero vuoto è -1
- Un **albero AVL** è un **albero binario bilanciato in altezza**

ESEMPIO: FATTORE DI BILANCIAMENTO

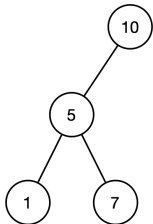


ESEMPIO: ALBERO SBILANCIATO

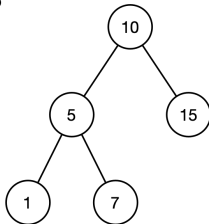


ALBERI AVL?

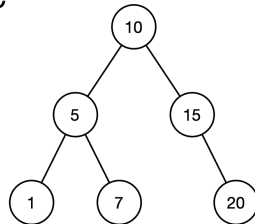
A



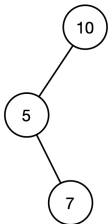
B



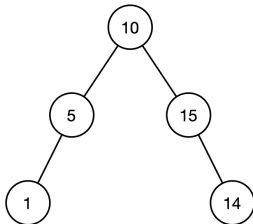
C



D



E

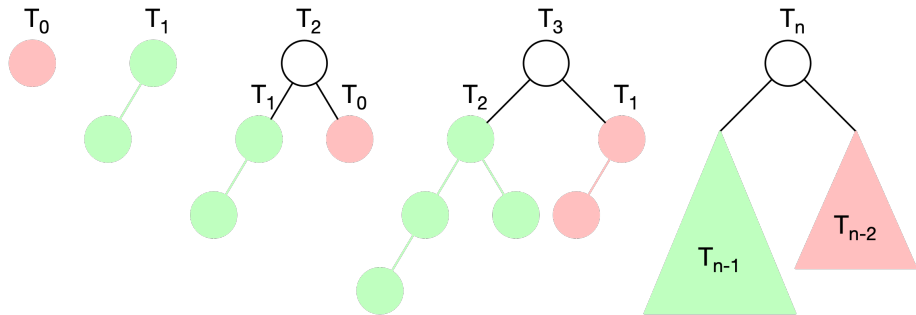


F



ALTEZZA DI UN ALBERO AVL

- Qual è l'altezza massima di un albero AVL?
- Valutiamo l'altezza dell'albero **bilanciato** con **sbilanciamento massimo**
- **Albero di Fibonacci**: per ogni nodo u non foglia $|\beta(u)| = 1$



N.B. Non è l'albero delle chiamate della **funzione ricorsiva di Fibonacci**

ALTEZZA DI UN ALBERO DI FIBONACCI

- Sia n_h il numero di nodi di un albero di Fibonacci di altezza h
- Per costruzione abbiamo che

$$n_h = n_{h-1} + n_{h-2} + 1$$

Teorema

Il numero di nodi di un albero di Fibonacci di altezza h è uguale a

$$n_h = F_{h+3} - 1$$

dove F_{h+3} è il $(h+3)$ -esimo numero di Fibonacci

- Ricordiamo che $F_1 = F_2 = 1$

n_0	n_1	n_2	n_3
$2 - 1 = 1$	$3 - 1 = 2$	$5 - 1 = 4$	$8 - 1 = 7$

ALTEZZA DI UN ALBERO DI FIBONACCI

■ Dimostriamo per induzione che $n_h = F_{h+3} - 1$

■ Casi base:

■ $n_0 = F_3 - 1 = 2 - 1 = 1 \Rightarrow \text{vero}$

■ $n_1 = F_4 - 1 = 3 - 1 = 2 \Rightarrow \text{vero}$

■ Caso induttivo: assumiamo vere

$$n_{h-1} = F_{h+2} - 1 \text{ e } n_{h-2} = F_{h+1} - 1$$

dobbiamo dimostrare che $n_h = F_{h+3} - 1$

$$\begin{aligned} n_h &= n_{h-1} + n_{h-2} + 1 \\ &= (F_{h+2} - 1) + (F_{h+1} - 1) + 1 \\ &= F_{h+3} - 1 \Rightarrow \text{vero} \end{aligned}$$

ALTEZZA DI UN ALBERO AVL

- Ricapitolando: un albero di Fibonacci di altezza h ha

$$n_h = F_{h+3} - 1 \text{ nodi}$$

- Ricordiamo che

$$F_h = \Theta(\phi^h) \text{ dove } \phi \approx 1.618$$

da cui otteniamo che

$$n_h = F_{h+3} - 1 = \Theta(\phi^{h+3}) = \Theta(\phi^h)$$

equivalentemente

$$\log_{\phi} n_h = \Theta(\log_{\phi} \phi^h) \Rightarrow \log_{\phi} n_h = \Theta(h)$$

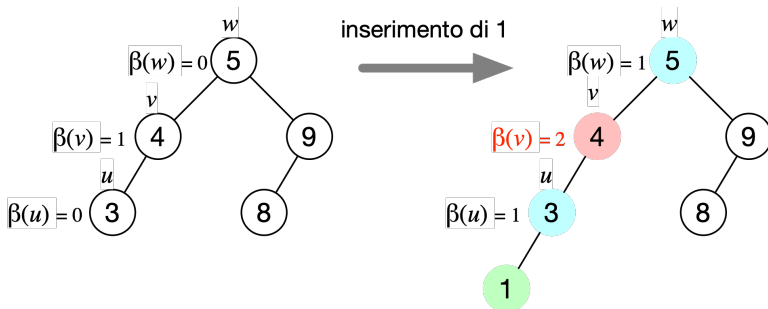
e possiamo quindi concludere che

$$h = \Theta(\log n_h)$$

- Poiché l'albero di Fibonacci con n nodi è l'albero con altezza massima tra tutti gli alberi binari bilanciati con n nodi possiamo concludere che l'altezza h di un albero AVL con n nodi è $\Theta(\log n)$

MANTENERE IL BILANCIAMENTO

- La ricerca su un albero AVL viene effettuata come su un BST
- Inserimenti e rimozioni invece richiedono di essere modificati per mantenere il bilanciamento dell'albero



- Per poter verificare il bilanciamento ogni nodo u deve mantenere informazioni sull'altezza del proprio sottoalbero $u.height$
 - Serve per poter calcolare il fattore di bilanciamento β

AGGIORNAMENTO ALTEZZA E BILANCIAMENTO

```
1: function UPDATEHEIGHT(Node  $u$ )
2:   if  $u \neq \text{NIL}$  then
3:      $\text{lefth} = -1, \text{righth} = -1$ 
4:     if  $u.\text{left} \neq \text{NIL}$  then
5:        $\text{lefth} = u.\text{left}.\text{height}$ 
6:     if  $u.\text{right} \neq \text{NIL}$  then
7:        $\text{righth} = u.\text{right}.\text{height}$ 
8:      $u.\text{height} = \text{MAX}(\text{lefth}, \text{righth}) + 1$ 
```

■ Entrambe costano $O(1)$

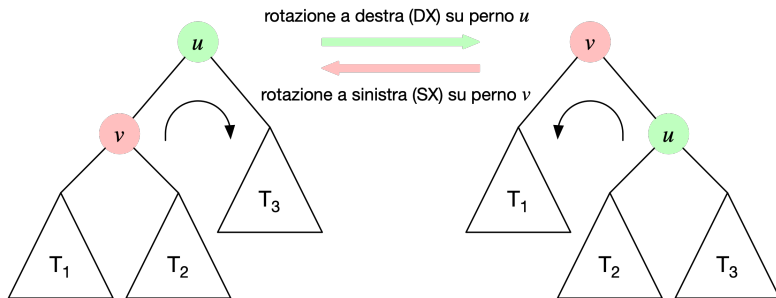
■ UPDATE aggiorna l'altezza del nodo u

```
1: function BALANCEFACTOR(Node  $u$ )  $\rightarrow \text{INT}$ 
2:    $\text{lefth} = -1, \text{righth} = -1$ 
3:   if  $u \neq \text{NIL}$  and  $u.\text{left} \neq \text{NIL}$  then
4:      $\text{lefth} = p.\text{left}.\text{height}$ 
5:   if  $u \neq \text{NIL}$  and  $u.\text{right} \neq \text{NIL}$  then
6:      $\text{righth} = p.\text{right}.\text{height}$ 
7:   return  $\text{lefth} - \text{righth}$ 
```

■ BALANCEFACTOR calcola il fattore di bilanciamento β del nodo u

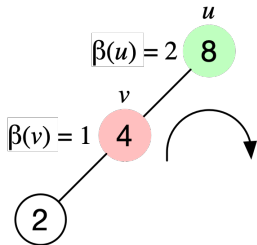
ROTAZIONI

- **Rotazione semplice:** operazione fondamentale per ribilanciare l'albero

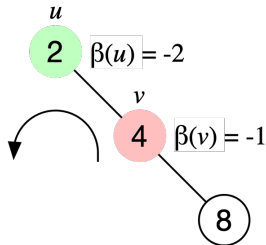
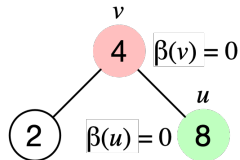
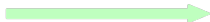


- Preserva la proprietà di ordine dei BST
 - La chiave $u.key \geq v.key$ e maggiore o uguale di ogni chiave in T_1, T_2
 - La chiave $v.key \leq u.key$ e minore o uguale di ogni chiave in T_2, T_3
- Una rotazione semplice modifica solo il fattore di bilanciamento di u, v

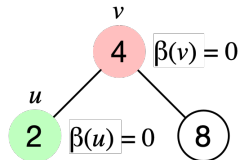
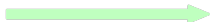
ESEMPI DI ROTAZIONI SEMPLICI



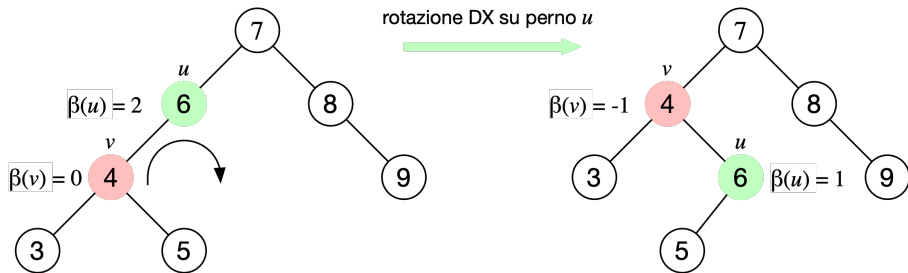
rotazione DX su perno u



rotazione SX su perno u



ESEMPIO DI ROTAZIONE SEMPLICE



PSEUDOCODICE: ROTAZIONE A DESTRA

```
1: function RIGHTROTATE(AVL  $T$ , NODE  $u$ )
2:   if  $u \neq \text{NIL}$  and  $u.\text{left} \neq \text{NIL}$  then
3:      $v = u.\text{left}$ 
4:      $u.\text{left} = v.\text{right}$ 
5:      $v.\text{right} = u$ 
6:     if  $u.\text{left} \neq \text{NIL}$  then
7:        $u.\text{left}.\text{parent} = v$ 
8:      $v.\text{parent} = u.\text{parent}$ 
9:     if  $u.\text{parent} == \text{NIL}$  then      ▷  $u$  was the root node
10:       $T.\text{root} = v$                   ▷  $v$  is the new root
11:     else if  $u.\text{parent}.\text{left} == u$  then ▷  $u$  was a left child
12:        $v.\text{parent}.\text{left} = v$ 
13:     else                             ▷  $u$  was a right child
14:        $v.\text{parent}.\text{right} = v$ 
15:      $u.\text{parent} = v$ 
16:     UPDATEHEIGHT( $u$ )
17:     UPDATEHEIGHT( $v$ )
```

■ Costo: $O(1)$

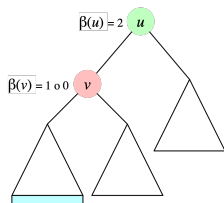
■ LEFTROTATE simmetrica

Sbilanciamenti

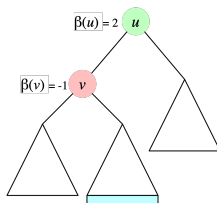
- Assumiamo di avere un albero AVL bilanciato, in cui il sottoalbero radicato in un nodo u diventa sbilanciato in seguito ad una operazione di inserimento o rimozione

- Abbiamo quattro casi (simmetrici a due a due)

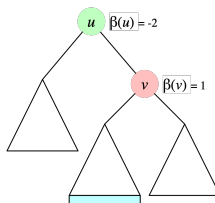
- Sbilanciamento SS: $\beta(u) = 2$ e $\beta(u.left) \geq 0$
- Sbilanciamento DD: $\beta(u) = -2$ e $\beta(u.right) \leq 0$
- Sbilanciamento SD: $\beta(u) = 2$ e $\beta(u.left) = -1$
- Sbilanciamento DS: $\beta(u) = -2$ e $\beta(u.right) = 1$



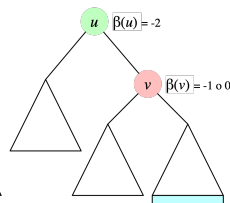
SS (Sinistro-Sinistro)



SD (Sinistro-Destro)

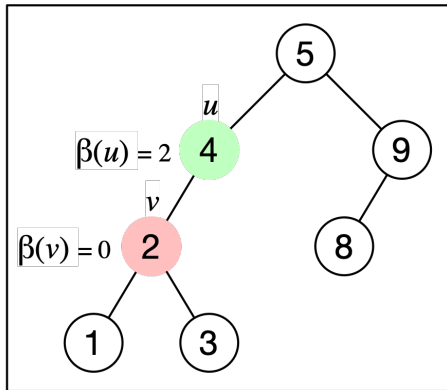
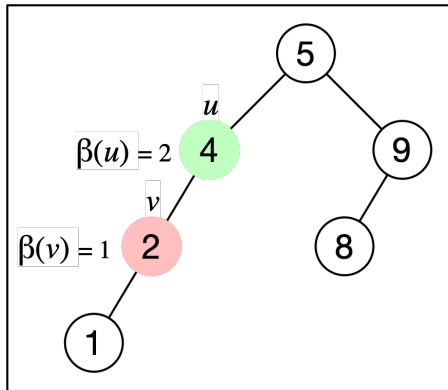


DS (Destro-Sinistro)

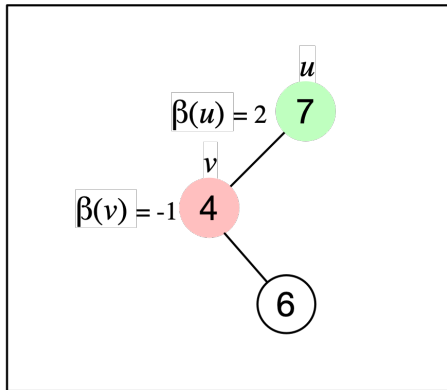
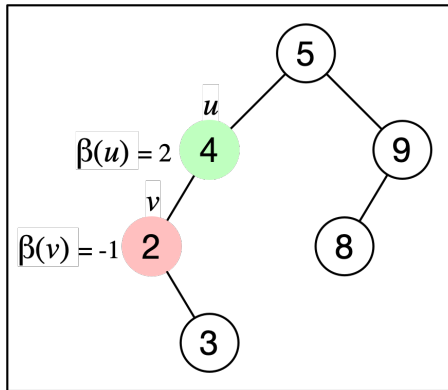


DD (Destro-Destro)

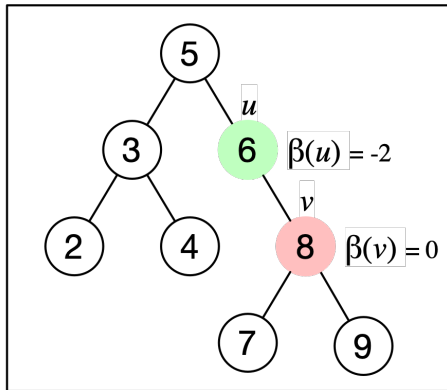
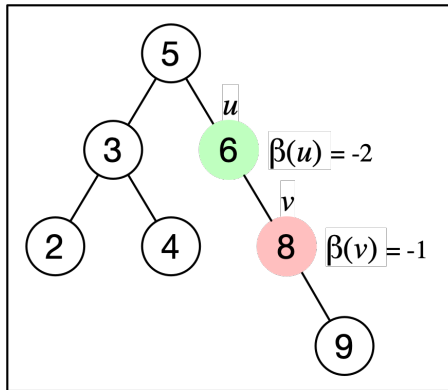
ESEMPI: SBILANCIAMENTI SS



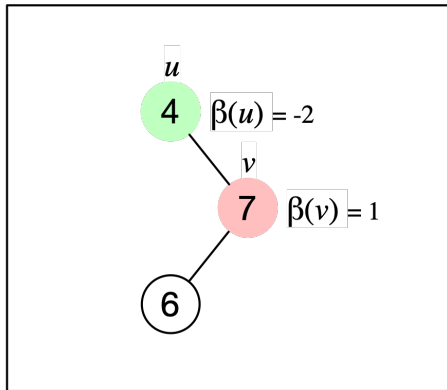
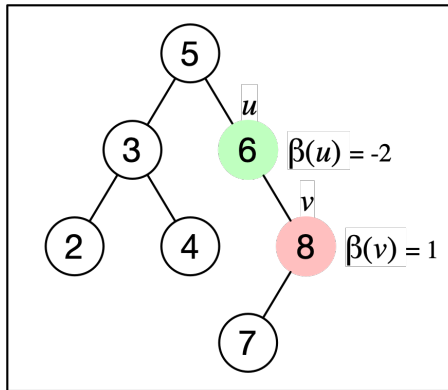
ESEMPI: SBILANCIAMENTI SD



ESEMPI: SBILANCIAMENTI DD



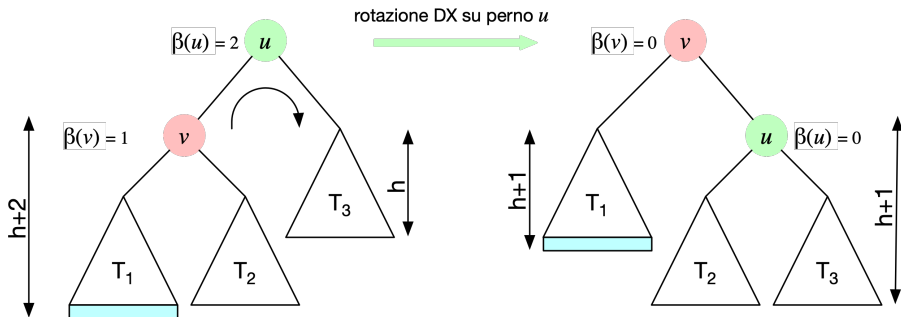
ESEMPI: SBILANCIAMENTI DS



SBILANCIAMENTO SS $\Rightarrow$ ROTAZIONE DX 1/2

- Ribilanciamento per uno sbilanciamento SS $\Rightarrow$ Rotazione DX

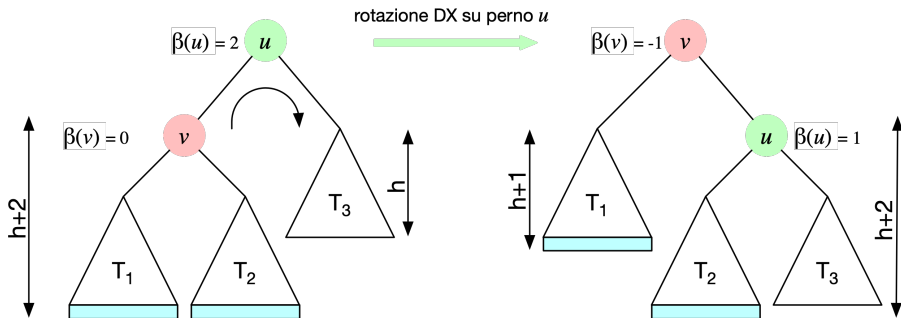
$$\beta(u) = 2 \text{ e } \beta(u.\text{left}) = 1$$



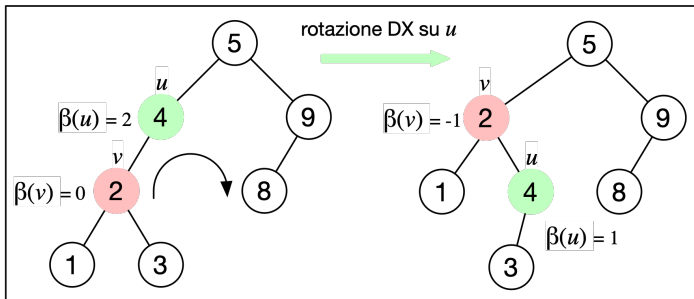
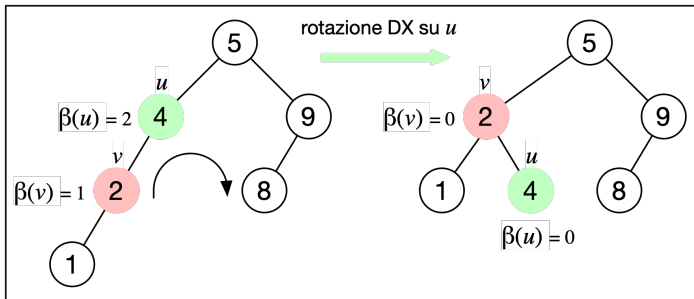
SBILANCIAMENTO SS $\Rightarrow$ ROTAZIONE DX 2/2

- Ribilanciamento per uno sbilanciamento SS $\Rightarrow$ Rotazione DX

$$\beta(u) = 2 \text{ e } \beta(u.\text{left}) = 0$$



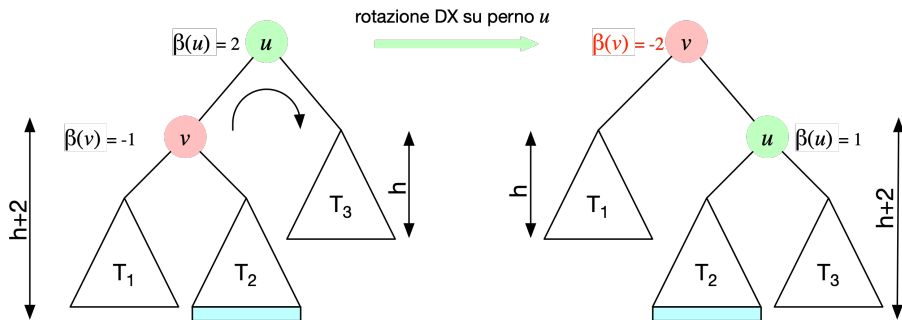
ESEMPI: SBILANCIAMENTO SS $\Rightarrow$ ROTAZIONE DX



SBILANCIAMENTO SD $\Rightarrow$ ROTAZIONE DX?

- Ribilanciamento per uno sbilanciamento SD con Rotazione DX

$$\beta(u) = 2 \text{ e } \beta(u.\text{left}) = -1$$



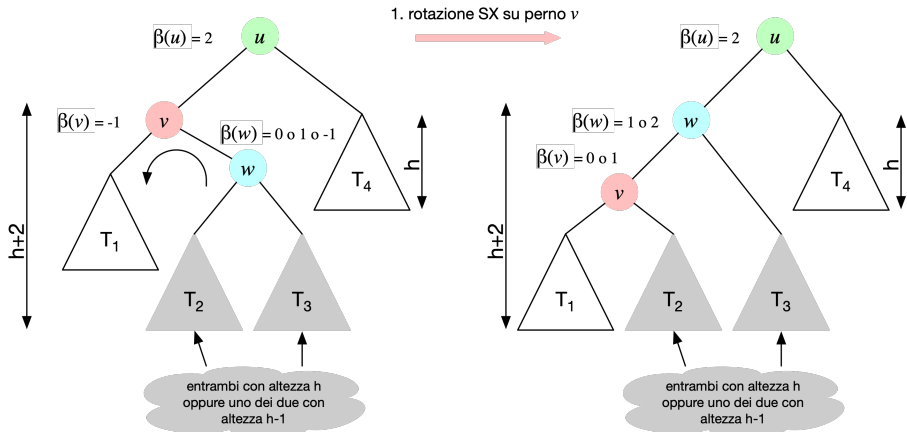
- Non funziona!!
- Abbiamo ottenuto uno sbilanciamento DS

SBILANCIAMENTO SD $\Rightarrow$ ROTAZIONE SXDX 1/2

- Ribilanciamento per uno sbilanciamento SD $\Rightarrow$ Rotazione SXDX

$$\beta(u) = 2 \text{ e } \beta(u.\text{left}) = -1$$

- Step 1: rotazione SX con perno $u.\text{left}$ $\Rightarrow$ sibilanciamento SS

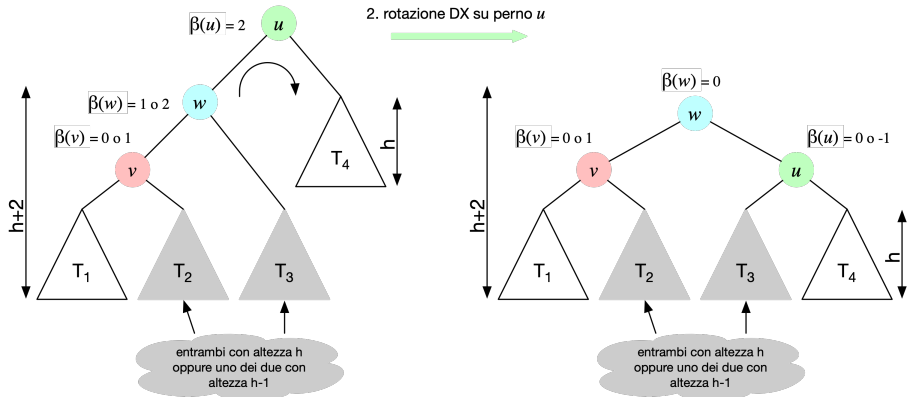


SBILANCIAMENTO SD $\Rightarrow$ ROTAZIONE SXDX 2/2

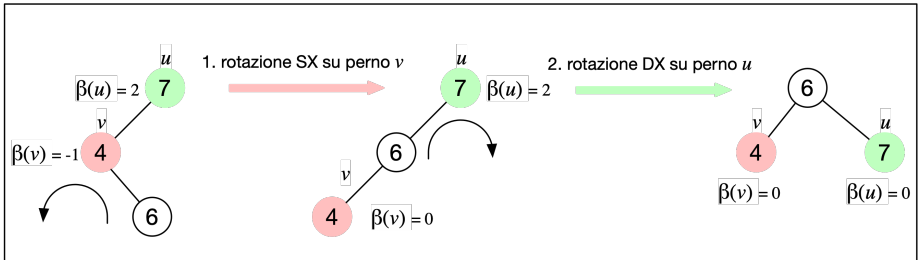
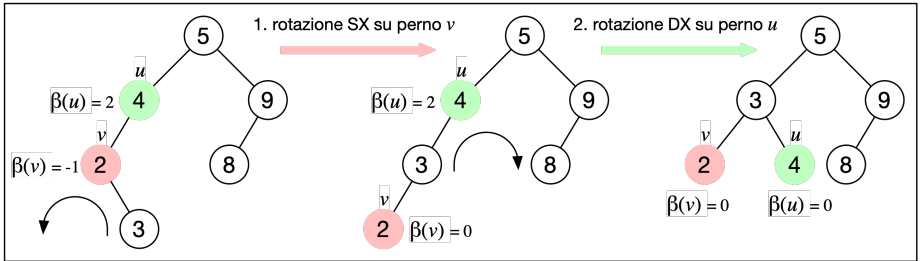
- Ribilanciamento per uno sbilanciamento SD $\Rightarrow$ Rotazione SXDX

$$\beta(u) = 2 \text{ e } \beta(u.\text{left}) = -1$$

- Step 2: rotazione DX con perno u



ESEMPI: SBILANCIAMENTO SD $\Rightarrow$ ROTAZIONE SXDX



RIASSUNTO: SBILANCIAMENTI E ROTAZIONI

- Sbilanciamento SS: $\beta(u) = 2$ e $\beta(u.left) \geq 0$
 - Rotazione DX su u
- Sbilanciamento DD: $\beta(u) = -2$ e $\beta(u.right) \leq 0$
 - Rotazione SX su u
- Sbilanciamento SD: $\beta(u) = 2$ e $\beta(u.left) = -1$
 - Rotazione SX su $u.left$
 - Rotazione DX su u
- Sbilanciamento DS: $\beta(u) = -2$ e $\beta(u.right) = 1$
 - Rotazione DX su $u.right$
 - Rotazione SX su u

PSEUDOCODICE: BILANCIAMENTO

```
1: function BALANCE(AVLTree  $T$ , NODE  $u$ )
2:    $b = \text{BALANCEFACTOR}(u)$ 
3:   if  $b == 2$  then
4:     if  $\text{BALANCEFACTOR}(u.\text{left}) == -1$  then
5:       LEFTROTATE( $T, u.\text{left}$ )
6:       RIGHTROTATE( $T, u$ )
7:   else if  $b == -2$  then
8:     if  $\text{BALANCEFACTOR}(u.\text{right}) == 1$  then
9:       RIGHTROTATE( $T, u.\text{right}$ )
10:    LEFTROTATE( $T, u$ )
```

■ Costo: $O(1)$

PSEUDOCODICE DI INSERT

```
1: function INSERT(AVLTree  $T$ , KEY  $k$ , DATA  $d$ )  $\rightarrow$  NODE
2:    $v = \text{BSTINSERT}(T, k, d)$ 
3:    $p = v.\text{parent}$ 
4:   while  $p \neq \text{NIL}$  and  $|\text{BALANCEFACTOR}(p)| \neq 2$  do
5:     UPDATEHEIGHT( $p$ )
6:      $p = p.\text{parent}$ 
7:   if  $p \neq \text{NIL}$  then
8:     BALANCE( $T, p$ )
9:   return  $v$ 
```

- BSTINSERT costa nel caso ottimo/medio/pessimo $\Theta(\log n)$
 - Ricordiamo che l'altezza dell'Albero AVL è $\Theta(\log n)$
 - Inoltre, l'albero è anche sempre bilanciato
- Il ciclo while (riga 4) viene eseguito $O(\log n)$ volte
 - Al peggio, per tutti i nodi sul percorso da v fino alla radice

DETTAGLI INSERIMENTO IN UN ALBERO AVL

- 1 Si inserisce il nuovo nodo come per i BST
 - Tutti i percorsi di inserimento sono lunghi $\Theta(\log n)$
 - Costo: $\Theta(\log n)$
 - 2 Si riaggiornano le altezze dei sotto-alberi, eventualmente cambiate
 - Nel caso peggiore, tale aggiornamento dovrà essere effettuato per tutti i nodi nel cammino dal nodo inserito fino alla radice
 - Costo: $O(\log n)$
 - 3 Se un nodo presenta fattore di bilanciamento ± 2 (nodo critico), occorre ribilanciare l'albero con la procedura di ribilanciamento
 - N.B. In caso di inserimento abbiamo al più un nodo critico
 - Dopo un ribilanciamento non continuiamo ad aggiornare altezze
 - Costo nel caso pessimo: $O(1)$
- Costo inserimento nel caso ottimo/medio/pessimo: $\Theta(\log n)$

PSEUDOCODICE DI DELETE

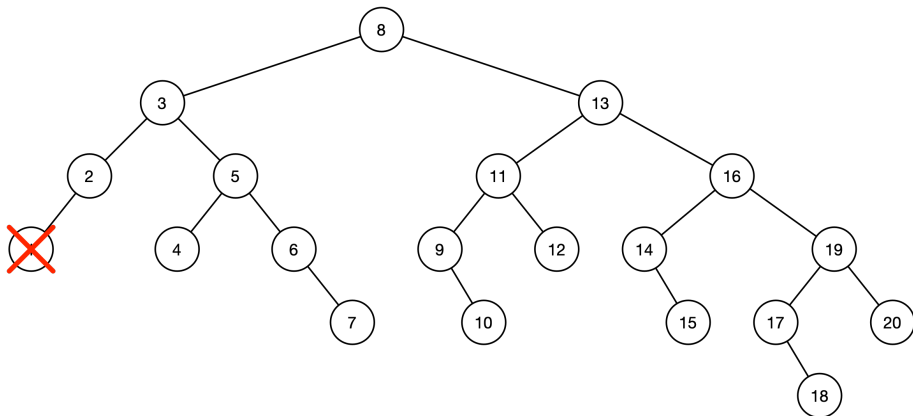
```
1: function DELETE(AVLTree  $T$ , KEY  $k$ , DATA  $d$ )
2:    $v = \text{BSTDELETE}(T, k)$ 
3:   if  $v \neq \text{NIL}$  then
4:      $p = v.\text{parent}$ 
5:     while  $p \neq \text{NIL}$  do
6:       if  $|\text{BALANCEFACTOR}(p)| == 2$  then
7:          $\text{BALANCE}(T, p)$ 
8:       else
9:          $\text{UPDATEHEIGHT}(p)$ 
10:       $p = p.\text{parent}$ 
11:   return  $v$ 
```

- BSTDELETE è la procedura DELETE su Alberi Binari
 - Ricordiamo che DELETE non azzerava il campo *parent* del nodo rimosso v , quindi l'assegnamento a riga 4 è corretto
- Notare che BALANCE può essere richiamata molte volte

RIMOZIONE IN UN ALBERO AVL

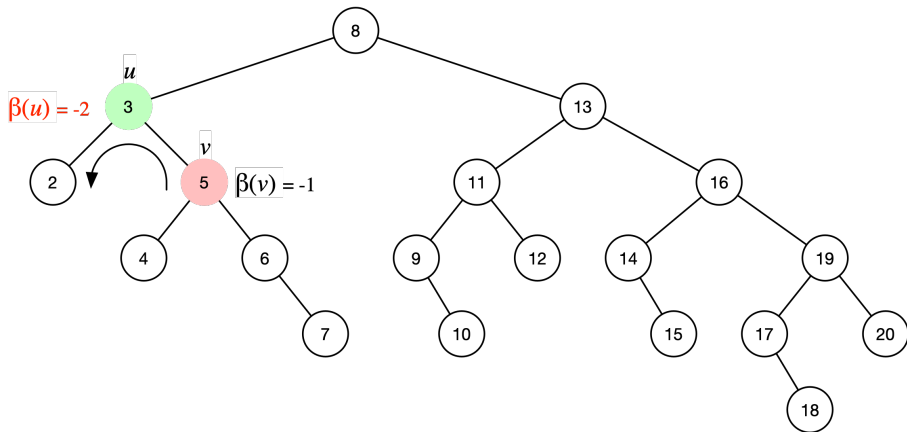
- 1 Si rimuove il nodo come per i BST (richiede una ricerca)
 - Costo nel caso pessimo: $\Theta(\log n)$
 - Costo nel caso medio: $\Theta(\log n)$
 - 2 Si riaggiornano le altezze dei sotto-alberi, eventualmente cambiate
 - Nel caso peggiore, tale aggiornamento dovrà essere effettuato per tutti i nodi nel cammino da una foglia fino alla radice
 - Costo nel caso pessimo: $\Theta(\log n)$
 - Costo nel caso medio: $O(\log n)$
 - 3 Se un nodo presenta fattore di bilanciamento ± 2 (nodo critico), occorre ribilanciare l'albero con la procedura di ribilanciamento
 - N.B. In caso di rimozione potremmo avere più nodi critici
 - Tutti i nodi critici sono in un unico percorso radice-nodo rimosso
 - Costo nel caso pessimo: $\Theta(\log n)$
 - Costo nel caso medio: $O(\log n)$
- Costo medio/pessimo della rimozione in un albero AVL: $\Theta(\log n)$

ESEMPIO: ROTAZIONI A CASCATA 1/4



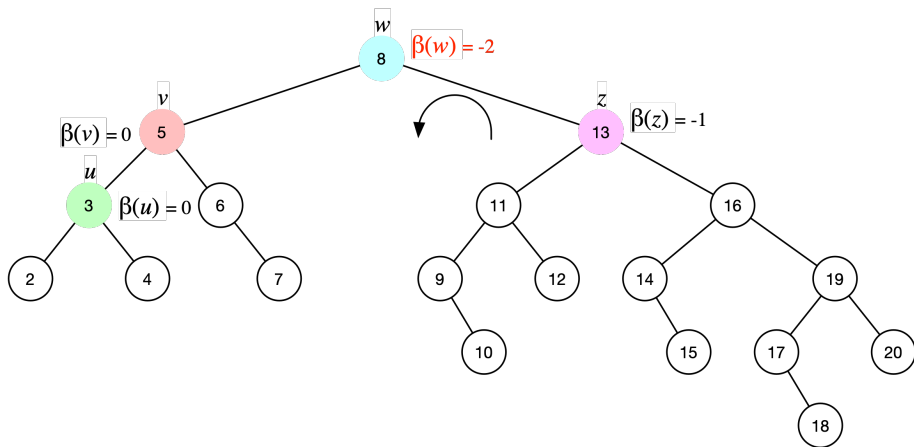
Albero AVL bilanciato prima della rimozione del nodo con chiave 1

ESEMPIO: ROTAZIONI A CASCATA 2/4



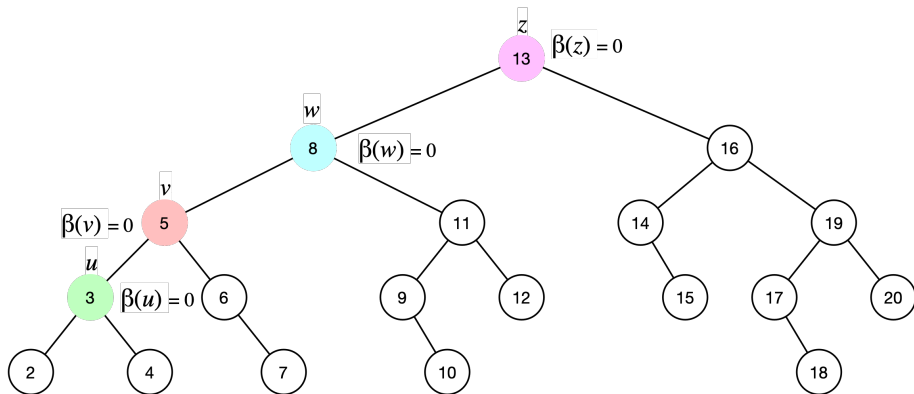
Ribilanciamento: rotazione SX con perno u

ESEMPIO: ROTAZIONI A CASCATA 3/4



Ribilanciamento: rotazione SX con perno w

ESEMPIO: ROTAZIONI A CASCATA 4/4



Albero ribilanciato

ALTRI ALBERI BILANCIATI

- Gli Alberi AVL non sono l'unica struttura dati auto-bilanciante
- Altre implementazioni non richiedono necessariamente alberi binari
- Tra le strutture dati maggiormente note abbiamo
 - B-alberi (Bayer e McCreight, 1970)
 - Alberi generici, non necessariamente binari
 - Utilizzati per database e file system
 - Alberi 2-3 (Hopcroft, 1970)
 - Ogni nodo intero ha 2 oppure 3 nodi
 - Tutte le foglie sono sempre allo stesso livello
 - Le chiavi sono tutte nelle foglie (e ordinate)
 - RB-Alberi o Red-Black Trees (Guibas e Sedgewick, 1978)
 - Alberi binari come gli AVL
- Tutte supportano inserimento, rimozione e ricerca in tempo logaritmico

DIZIONARIO: RIASSUNTO DEI COSTI

	SEARCH		INSERT		DELETE	
	Medio	Pessimo	Medio	Pessimo	Medio	Pessimo
Array non ordinati	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Array ordinati	$O(\log n)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$	$O(n)$	$\Theta(n)$
Lista concatenata	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Albero Binario di Ricerca	$\Theta(\bar{h})$	$\Theta(h)$	$\Theta(\bar{h})$	$\Theta(h)$	$\Theta(\bar{h})$	$\Theta(n)$
Albero AVL	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$

- h = altezza dell'albero, $\bar{h}$ = altezza media dell'albero
- L'implementazione di un dizionario con Alberi AVL richiede unicamente la modifica dell'operazione di inserimento per evitare chiavi ripetute (esattamente come abbiamo discusso per gli Alberi Binari di Ricerca)
- Gli Alberi AVL assicurano un costo logaritmico per tutte le operazioni